

Cognito

Learn, Teach, Verify, Improve

THEME

Reinventing Education with AI

DOCUMENT

The idea, the problem it addresses, and how AI is used to solve it

DATE

October 2026

Riya passes every quiz in her intro programming course. But when a friend asks her to explain recursion, she can't. She never noticed the gap, her teacher never saw it, and the final exam will show it only when it's too late to fix.

COGNITO IN ONE SENTENCE

Students learn a concept, then teach it to an AI called Bit. How well they teach shows what they really understand, and that turns into a clear plan for the teacher's next class.

SOLUTION AT A GLANCE

- **Learn through what you love.** Cognito explains concepts using examples from a student's interests, such as cricket, movies, space or gaming. Beginners start with guided examples, so the app doesn't assume any prior knowledge.
- **Teach Bit to reveal your thinking.** Bit plays a curious learner who asks simple questions and sometimes offers an explanation that sounds right but is wrong. As the student teaches Bit, an AI assessor checks their reasoning against a checklist for the concept. It records the exact words that show understanding or a possible misconception.
- **Repair the gap, then return.** If a student struggles, Cognito checks an earlier concept that this one depends on, and gives a short revision activity if it's needed. The student then returns to the concept they were learning.
- **Verify understanding independently.** A Capstone Check uses fresh questions with no hints, chosen from the student's concept map. Its results confirm or correct the progress shown on the map. Testing becomes part of learning, and students and teachers can see what needs attention and why.
- **Give teachers a clear next step.** The dashboard shows common misconceptions, how many tested students each one affects, and the evidence behind each finding. A Next-Class Brief suggests what to cover, an example to show and a question to check understanding afterwards.

01 The problem

The theme asks what changes for students, for teachers and for how learning is measured. Today all three break down in the same place: **when a student is confused, nobody turns that confusion into a clear next step.**

	What happens today	Why it matters
Students	The whole class gets the same explanation at the same speed, even though each student has different gaps and interests. When a student struggles, the real cause is often an earlier topic, and they may not know which one, or may be too shy to ask.	A student can feel they understand just because an explanation looks familiar. But being able to repeat an answer doesn't mean they can explain it, use it in a new problem or remember it a week later.
Teachers	They see marks, not the thinking behind them. With 30 to 100+ students, they can't find out what each one is confused about.	They end up re-teaching the wrong thing, or the right thing to the wrong students.
Measurement	One big exam at the end gives one number, which doesn't say <i>what</i> the student misunderstood.	Gaps are found too late to fix, and nobody checks whether students still remember the topic later.

02 The idea: one loop, four pillars

STUDENT LOOP



TEACHER LOOP

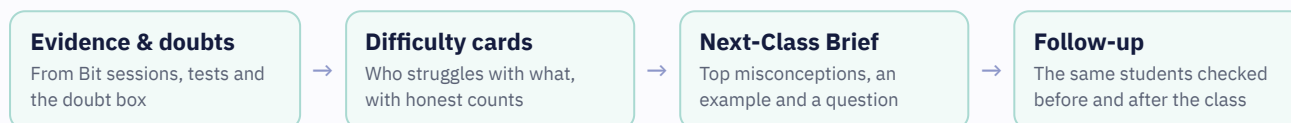


Figure 1. The Cognito loop. Steps 2 and 3 are the core, and everything a student does feeds the teacher loop.

THE CORE OF COGNITO

Teaching Bit, and the concept map built from that teaching, are the heart of Cognito. Everything else is there to support this loop.

Pillar	What's in it
Core loop	Learn → Teach Bit → See your understanding on the map → Fix gaps → Prove it
Personalisation	Interest profile, interest-based lessons, projects set by the teacher
Levels	Bit's style and difficulty settings, and revision levels from 1 up to Grandmaster
Retention	Spaced reviews (a green concept slowly fades to yellow) and streaks
Teacher insights	Student doubts on the dashboard, common difficulties, the Next-Class Brief and a follow-up check

03 How students learn: the Cognito framework

Students first learn each concept in the app's **Learn mode**, which explains it through their own interests with a worked example and study notes. They then **teach it to Bit**, so their gaps show up and get fixed, and **prove it in the Capstone Check**. The teacher spends class time only on the **misconceptions most of the class shares**, guided by the Next-Class Brief.

Each concept goes through six stages:

Stage	What the student does	How Cognito helps	Why it works
1. Learn	Reads a short lesson built around their interest, follows a worked example, and uses the study notes	The Tutor explains step by step. Buttons ask for a <i>simpler</i> version, an <i>analogy</i> or the <i>code</i> .	New ideas stick better when they're linked to something familiar
2. Try	Answers one small guided question	Gives instant feedback. The student can ask a doubt at any time.	Checks the first understanding straight away
3. Teach	Explains the concept to Bit in their own words and answers Bit's questions	Bit asks "why?" and makes mistakes on purpose. The Assessor records what the student understood.	You only truly know something when you can explain it

Stage	What the student does	How Cognito helps	Why it works
4. Fix	If a gap shows up, answers two quick questions on the earlier concept, revises it and comes back	The concept map finds the missing earlier concept	Fixes the cause of the problem, not just the symptom
5. Prove	Takes the Capstone Check: fresh questions, no hints	Picks the questions from the student's own map	Shows the student can do it alone
6. Use and remember	Does a project set by the teacher, and comes back for short reviews over the following days	Schedules reviews after 1, 3, 7, 14 and 30 days, with levels up to Grandmaster	Using knowledge and recalling it later is what makes it last

04 One class, many speeds: what does the teacher teach?

If every student learns at their own speed, what happens in class? **Everyone works on the same chapter, but the teacher spends class time only where it's needed.**

- **Same chapter, same deadline, different speeds.** The teacher picks the chapter and a time window (for example, "*Recursion, this week*"). Every student works through the same concepts in the app, at their own speed. The class moves on to the next chapter together, so nobody is left behind and nobody runs too far ahead alone.
- **The app handles first contact with each concept.** Students learn and teach Bit in the app, as homework or in lab time, so the teacher doesn't have to repeat the whole lecture.
- **In class, the teacher covers what most students got wrong.** The Next-Class Brief lists the top two or three misconceptions shared across the class, and the teacher explains those to everyone at once.
- **Groups handle everyone else.** The dashboard sorts students into three groups: **needs help**, **on track** and **ahead**. The teacher sits with the needs-help group. On-track students keep learning in the app, and students who are ahead try harder levels or a project.

A sample 40-minute class:

Time	Who	What happens
0–10 min	Whole class	The teacher explains the top misconception from the brief, using the suggested example
10–30 min	Groups	The teacher helps the "needs help" group. Others continue in the app or try a harder level or project.
30–40 min	Whole class	Everyone answers the follow-up question, and the results go straight to the dashboard

Slower students are spotted early, faster students always have something challenging, and the teacher's time goes where it helps most.

05 Built from scratch, not bolted on

In a normal classroom, learning and testing are separate, and feedback arrives weeks later. **In Cognito they are the same activity.** Every explanation a student gives Bit counts as evidence, and every piece of evidence leads to a next step. Copying an answer doesn't work, because Bit keeps asking follow-up questions. The exam still exists, but **it confirms what the app has already seen instead of being the only verdict.** AI explains and asks questions, teachers keep making the classroom decisions, and students stay active.

06 Meet Bit, and the Assessor behind Bit

Bit is an AI study buddy who acts like a curious beginner, and the student's job is to teach Bit. Research on the *protégé effect* shows that students try harder and learn more when they have to teach someone else (Chase et al., 2009). Explaining something out loud also exposes gaps that rereading hides.

*Example topic: throughout this document we use **recursion** as the example. Recursion is a beginner programming idea where a function solves a problem by calling a smaller version of itself, and it needs a "base case" to know when to stop. The same approach works for any topic.*

How Bit behaves

- **One question at a time.** Bit asks simple questions, asks for examples and pushes back when an explanation is incomplete: *"Why does that call finish first?" "What if the number is zero?"*
- **Gets things slightly wrong.** Bit repeats the idea back with a small mistake, and the student has to notice it and correct it.
- **Spot Bit's mistake.** Sometimes Bit confidently says a common wrong idea: *"So when we reach the base case, that's the final answer and everything stops, right?"* If the student agrees, the misconception is revealed. If the student corrects Bit, they've shown real understanding.
- **Style and difficulty settings.** The student chooses Bit's personality (**curious beginner**, **skeptical friend** or **tough interviewer**) and, separately, how hard Bit's questions are (Level 1 to 3). Neither setting makes marking easier or harder.
- **Help when stuck.** Bit helps in small steps: **a hint question → a link to the right part of the notes → an analogy → part of the explanation, only as a last resort.** Each hint used is recorded, because needing help is useful information too.
- **Bit never does the student's work.** If a student asks Bit for the answer, Bit says *"Hey, I'm the one learning here!"* and points them to Learn mode.
- **Bit's notebook.** Bit writes down what it has been taught (*base case ✓, recursive step ✓, getting closer to the base case ✗*). At the end of a session, a short card shows what the student explained well, what tripped them up and one thing to try next.

The hidden Assessor. A second AI works behind Bit and **never talks to the student.** After each explanation, it checks what the student said against the concept's checklist (the rubric). It records what they got right, any likely misconception, and **the exact words that show it.** Bit is friendly and chatty, while the Assessor is strict and consistent. Keeping them apart means the friendly one never marks and the marker never chats.

A sample conversation (Riya picked *cricket*):

Riya: Recursion is like bowling an over. To bowl six balls, you bowl one ball, and then you only have to bowl the rest of the over, which is five balls.

Bit (*skeptical friend*): Okay, but when does it stop? Do we keep bowling forever?

Riya: It stops when there are no balls left. Then the over is done.

Bit: Oh, so "no balls left" is the base case! (*notebook: base case ✓*) What would happen if we forgot it?

What Bit means for teachers. Bit lets a teacher learn from dozens of private explanations without reading any of them. Each one becomes a short note with the concept, the misconception and the student's own words. The dashboard then shows which concepts students can't explain, and why.

07 The student journey

7.1 Sign-up

Students pick their interests (**cricket, movies, space, gaming**) and how they want to start. **"I'm new"** starts with a lesson, so a complete beginner is never stuck. **"Check what I know"** starts with a short quiz instead.

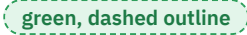
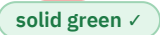
7.2 Learn

For each concept, the Tutor gives a short explanation, one worked example and a quick question, all based on reviewed course notes. One-tap buttons ask for **an analogy**, **a simpler version** or **the code**. The Tutor explains, but it never does the student's work or answers its own questions. Explanations use the student's interest:

Interest	Recursion explained as...
Cricket	Bowling an over: bowl one ball, and what's left is the same job with one ball fewer. When no balls are left, the over is done.
Movies	A ticket queue at the cinema: to find your place, ask the person in front. They ask the person in front of them, and so on. The first person says "I'm 1st", and the answers come back down the line with each person adding one.
Space	A launch countdown: say "10", then count down from 9, and keep going until you reach zero and shout "Liftoff!"
Gaming	A treasure chest with a smaller chest inside, and a smaller one inside that. You keep opening chests until the last one, which holds the treasure.

Each analogy ends with **"Where this analogy breaks"**. For example: *"In a real over, the bowler just bowls the next ball. In recursion, each step also waits for the smaller job to finish and then hands its answer back."* If an analogy would confuse more than it helps, the student gets a plain explanation instead.

7.3 Teach Bit, and watch the map update

The student explains the concept to Bit (Section 6). Their **personal concept map** updates as they go. Each concept is shown as  (a gap or misconception),  (shaky),  (explained well but not yet tested) or  (passed the test). Clicking **"Why this score?"** on any concept shows the student's own words and the reason behind its colour.

7.4 Fix: go back to the real gap

When a concept turns red, Cognito doesn't make the student redo the whole chapter, and it doesn't assume every earlier topic is weak. It follows the concept map back to the most likely missing piece (for example, *how a function returns a value*) and asks two quick questions on it. If that is the gap, the student revises it and then **comes back exactly where they left off**. If the earlier topic is fine, Cognito looks more closely at the current concept instead.

7.5 Prove it: the Capstone Check (the big exam, redesigned)

When the student is ready, they take a short test called the **Capstone Check, built from their own concept map**. Weak concepts (red or yellow) get two questions and strong ones get one spot-check. There are **no hints and no Bit**. Some questions ask "what does this code print?", some ask the student to explain why, and some ask them to **find the mistake in a wrong explanation**. Before submitting each answer, the student taps how sure they are: *sure / think so / guessing*. A confident wrong answer points to a misconception, not just a gap. Fixing a confident mistake also tends to stick better (Butterfield & Metcalfe, 2001).

The test updates the map; it doesn't replace it. Passed concepts become "tested". If the map and the test disagree, Cognito shows the mismatch (see Section 11).

7.6 Guess first, then watch it run

Before the app runs a small piece of code, such as a countdown from 3, the student **guesses what it will print, in order**. An animation then runs the code step by step, showing each call starting and finishing. If the guess was wrong, the app **highlights the exact step where the student's thinking went off track**. This catches students who memorised a rule without really understanding it.

7.7 Use it and remember it

- **Revision levels:** Level 1 Basic → Level 2 Application → Level 3 Challenge → **Grandmaster**. The Grandmaster level is a debugging task that the student defends against tough-interviewer Bit. Passing it earns the **Grandmaster title for that topic**, for example *"Recursion Grandmaster"*.
- **Projects:** when a concept is tested and passed, the student gets a small project to apply it. **Projects are set by the teacher**, so they fit the class and the facts are right. Teachers can match them to interests, for example a cricket scoreboard countdown, a cinema-queue counter, a rocket-launch timer or a treasure-chest game.
- **Spaced reviews:** people forget over time (the "forgetting curve"). So a tested green concept **slowly fades back to yellow**, and the app asks for a quick review after 1, 3, 7, 14 and 30 days. A short re-teach to Bit plus one fresh question turns it green again.
- **Streaks** count days with real learning activity, not logins. They show effort, **never proof of learning**.

08 What changes for teachers

- **Doubts reach the dashboard.** Students can ask the AI any doubt. The AI answers it and saves a short note for the teacher: the concept, the difficulty and the student's words. The app tells students clearly that **doubts are shared with their teacher, without their name unless they choose to add it**. That way it doesn't feel like being watched, and quiet students still get heard.
- **What most students struggle with.** Ranked cards show honest counts. They **separate students who said they were confused from students whose test answers show the gap**.

Example:

"The base case is the final answer": 9 of 24 tested students (6 were confident), 4 more said they were confused, 2 have fixed it after revision. [Open evidence](#) →

- **Class heatmap.** The same concept map, coloured by how much of the class has passed each concept, so the teacher can see where the class is stuck at a glance.
- **The Next-Class Brief.** One click produces a half-page plan:

Next class: Recursion (Section B, 24 of 31 tested)

1. "The base case is the final answer" (9 students). *Show:* work through `factorial(3)` on the board with one sticky note per call, and ask "Where does the answer of `factorial(1)` go next?"
2. "The function never gets closer to stopping" (5 students). *Show:* a countdown that forgets to subtract 1 and runs forever, then fix it together.
3. "All the calls share the same variable" (4 students). *Show:* draw a separate box for each call, each with its own value of `n`.

Check afterwards: "What does `mystery(3)` print?" (from the reviewed question bank)

- **Did it work?** After the class, the follow-up question compares **the same students before and after**. Students who haven't answered are listed separately, and the AI never claims the lesson *caused* the improvement.
- **The misconception list grows.** Doubts that don't match any known misconception are grouped and suggested to the teacher as new entries. The teacher approves each one before it's added.

09 How learning gets measured

Cognito measures **what students can do on their own, how long they remember it, and whether teachers can act on the results.**

What we measure	How we measure it	What it tells us
Understanding on their own	Similar questions before learning and after, with no hints	Whether the student can do it without help
Ability to explain	The same checklist every time: correct, well-reasoned, good examples, handles follow-up questions	Whether the student can explain <i>how</i> and <i>why</i> it works
Fixing misconceptions	After revision, a <i>different</i> question on the same idea	Whether that specific mistake is really gone
Using it in new places	A new kind of problem, or a project	Whether understanding goes beyond memorised examples
Remembering	Fresh questions days later (spaced reviews)	Whether the learning lasts
Needing less help	We count how many hints the student needs on similar questions over time. The number should go down.	Whether the student is becoming independent
Trusting the AI's marking	Teachers mark a sample of student explanations by hand, and we check how often the Assessor agrees. We also check whether students the map shows as green actually pass the Capstone Check.	Whether the AI's judgement can be trusted
Usefulness for teachers	Teachers rate the briefs and the difficulty cards, and report the time they save	Whether the insights are accurate and useful
Effect of the class	The same students' answers before and after the class	Whether the difficulty went down
Keeping a routine	Sessions, reviews done, return visits	Whether students keep learning (effort, not proof of learning)

10 How AI is used, and what it never does

AI helper	What it does	What it never does
Tutor	Explains concepts through the student's interest, gives worked examples, answers doubts and writes the session card	Answer its own quiz questions or do the student's work
Bit	Has the teaching conversation: asks questions, makes small mistakes on purpose and gives step-by-step hints	Give the correct answer or mark anything
Assessor (hidden)	Checks each explanation against the concept's checklist, and records the student's words, any likely misconception and the earlier concept that might be missing	Talk to the student
Check grader	Marks the "explain why" answers in the Capstone Check, using the same checklist	Mark questions with one right answer, such as multiple choice or "what does this print?". Normal code marks those instantly, the same way every time.
Teacher helper	Summarises the class's difficulties and doubts, and writes the Next-Class Brief and the follow-up summary	Make up numbers. Every count is calculated from the saved results.

The learning content (concepts, the order to learn them, checklists, common misconceptions and notes) is drafted with AI help, then **checked by hand** before use, and teachers can edit it. Map colours, review dates, streaks and class statistics are all **calculated by normal code from saved results**, never guessed by the AI.

11 What sets Cognito apart

- 1. The student teaches, and the AI learns.** Most AI tutors leave the student passive. Bit makes the student do the thinking, and planted mistakes test whether the understanding is real.
- 2. Teaching is the measurement.** Evidence comes from how students explain things, and "Why this score?" always shows the student's own words.
- 3. The test also checks the system.** The Capstone Check is built from each student's own map. If the map says a student understands a concept but they fail the Capstone question on it (or the other way round), Cognito **shows that mismatch instead of hiding it**. The student goes back to that concept, and we learn where the Assessor needs to improve.
- 4. It fixes the cause, not the symptom.** Cognito goes back to the missing earlier concept and then returns the student to where they were, instead of making them reread the whole chapter.
- 5. Every student learns at their own speed, and the class still moves together.** The app does the first teaching, and the teacher spends class time only on the mistakes most students share.
- 6. Interests run through everything.** Lessons, Bit's examples, projects and reviews all use the student's interests, and every analogy says where it stops working.
- 7. Teachers get an action, not a percentage.** Instead of "class mastery: 62%", they get: "9 students believe X, here's the evidence, here's what to do tomorrow, and here's how to check it worked."
- 8. Safe to ask.** Private doubts reach the teacher without names, so quiet students count too.